

SIMATS ENGINEERING
ASSESSMENT TOOL 6 - LOW
(DESIGN TASK QUESTIONS)

COURSE CODE: ITA04

COURSE NAME: STATISTICS WITH R PROGRAMMING

COURSE OUTCOME COVERED

CO2: *Implement looping constructs, control structures, and recursive functions in R to solve computational problems with efficient program logic. (BL3, BL4)*

Assessment Tool Weightage: 15%

QUESTIONS: 20

Total Marks: 100

ANNEXURE A

DESIGN TASK QUESTIONS

Code of Conduct:

I [Aryan.ks](#) / 192325051 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A handwritten signature in black ink on a light gray background. The signature is written in a cursive style and appears to read 'Aryan'.

DESIGN TASK QUESTIONS

Task 1

Design a function named `build_robust_matrix` that takes a numeric vector and dimensions `r` and `c`. The function must populate an $(r \times c)$ matrix row-wise. If the vector runs out of values, it must reuse values from the beginning of the vector using a custom programmatic loop index rather than R's built-in recycling mechanism.

```
build_robust_matrix <- function(v, r, c) {  
  m <- matrix(0, r, c)  
  k <- 1  
  for(i in 1:r){  
    for(j in 1:c){  
      m[i,j] <- v[k]  
      k <- k + 1  
      if(k > length(v)) k <- 1  
    }  
  }  
  m  
}
```

Task 2

Create a script structure that implements a basic simulation loop. The loop must initialize an asset value at (100) and apply a random percentage change at each step. Use a repeat loop that breaks immediately if the asset drops below (50) or exceeds (200) , and prints the total number of iterations completed.

```
asset <- 100  
count <- 0  
  
repeat{  
  asset <- asset * (1 + runif(1, -0.1, 0.1))  
  count <- count + 1  
  if(asset < 50 || asset > 200) break  
}  
  
print(count)
```

Task 3

Design a custom filtering function called `validate_bounds` that accepts a vector `x` and two boundary scalars, `lower` and `upper`. The function must replace elements falling outside the boundaries with a string "Out of Bounds" and leave matching ones intact, outputting a mixed-type or coerced character vector without relying on standard library functions.

```
validate_bounds <- function(x, lower, upper){  
  out <- character(length(x))  
  for(i in seq_along(x)){  
    if(x[i] < lower || x[i] > upper)  
      out[i] <- "Out of Bounds"  
    else  
      out[i] <- as.character(x[i])  
  }  
  out  
}
```

Task 4

Build a fallback configuration structure using functional default values. Design a function `process_log` where the second argument `logger_type` defaults to a dynamic expression checking if a global logging variable exists. If it does not exist, initialize a default configuration structure locally.

```
process_log <- function(log_data,  
  logger_type = if(exists("global_logger"))  
    global_logger  
  else  
    list(type="default", level="INFO")){  
  list(data=log_data, logger=logger_type)  
}
```

Task 5

Design a complex multi-metric pipeline function named `generate_report`. It must accept a data frame, perform distinct aggregations (mean, standard deviation, and missingness metrics), and bundle these values along with a system timestamp into a structured, named list container where each value can be fetched securely by key.

```
generate_report <- function(df){  
  list(  

```

```

mean = sapply(df, mean, na.rm=TRUE),
sd = sapply(df, sd, na.rm=TRUE),
missing = sapply(df, function(x) sum(is.na(x))),
timestamp = Sys.time()
)
}

```

Task 6

Create a functional matrix lookup routing framework. Define a list of functions where each element corresponds to a specific geometric transformation (e.g., rotate_90, reflect_x). Design a primary router function that takes a matrix and a string name to fetch and execute the requested operation dynamically using bracket extraction notation.

```

ops <- list(
  rotate_90 = function(m) t(m[nrow(m):1,]),
  reflect_x = function(m) m[nrow(m):1,]
)

```

```

router <- function(mat, op){
  ops[[op]](mat)
}

```

Task 7

Design a recursive directory parsing emulation function named traverse_nodes. The function must simulate moving down a nested taxonomy tree structure by processing a list of lists. It must count the total leaf values present, implementing a hard-coded depth limit condition to prevent memory allocation faults on deep trees.

```

traverse_nodes <- function(x, depth=0, limit=10){
  if(depth > limit) return(0)
  if(!is.list(x)) return(1)
  total <- 0
  for(i in x)
    total <- total + traverse_nodes(i, depth+1, limit)
  total
}

```

Task 8

Build an element-wise matrix matching verification engine. The program must accept two matrices of identical dimensions and check whether every corresponding position satisfies an inequality rule ($(A_{ij} > B_{ij})$). It must return a single scalar boolean answer using strict non-short-circuiting element operators.

```
verify_matrix <- function(A, B){  
  all(A > B)  
}
```

Task 9

Design a mathematical series expansion module that handles vector limits. Create a function `evaluate_series` that iterates through numbers up to an input limit (N) . The function must drop any components divisible by (3) or (7) from the expansion series, accumulating the remaining sum using a next pipeline structure.

```
evaluate_series <- function(N){  
  s <- 0  
  for(i in 1:N){  
    if(i %% 3 == 0 || i %% 7 == 0) next  
    s <- s + i  
  }  
  s  
}
```

Task 10

Create a safe mathematical division system function `safe_divide` that takes two vectors. The function must verify indices before processing. If any denominator value equals zero or evaluates to missing (NA), it must set the output index result to a fallback code (-999) while keeping the rest of the computed vector intact.

```
safe_divide <- function(a, b){  
  out <- numeric(length(a))  
  for(i in seq_along(a)){  
    if(is.na(b[i]) || b[i] == 0)  
      out[i] <- -999  
    else  
      out[i] <- a[i] / b[i]  
  }  
}
```

```
  out
}
```

Task 11

Design a custom sliding window averaging loop for a numeric vector. The function must accept an array and a window size (W) , checking and executing an explicit error escape condition if the input array length is shorter than (W) . It must output a vector containing successive localized window averages.

```
sliding_average <- function(x, W){
  if(length(x) < W) stop("Window size exceeds vector length")
  out <- numeric(length(x) - W + 1)
  for(i in 1:(length(x)-W+1)){
    out[i] <- mean(x[i:(i+W-1)])
  }
  out
}
```

Task 12

Create a dynamic argument evaluation function `compute_power` that takes a base number x and an exponent p . Design the function so that if p is missing, it dynamically evaluates the mean value of a global variable array to use as the default exponent during execution.

```
compute_power <- function(x, p = mean(global_array)){
  x ^ p
}
```

Task 13

Design a functional control pipeline called `apply_stepwise`. The pipeline must receive a raw value vector and an ordered list containing three distinct mathematical function objects. It must iterate through the function list, passing the mutated vector output from one function step directly into the next.

```
apply_stepwise <- function(vec, fun_list){
  result <- vec
  for(f in fun_list){
    result <- f(result)
  }
  result
}
```

Task 14

Build a nested indexing search engine over a data frame. The loop must evaluate row entries across two different metrics columns. If a row satisfies one condition but fails a second tracking rule, it must map that specific coordinate into an error-tracking list returned at completion.

```
search_errors <- function(df){  
  errors <- list()  
  k <- 1  
  for(i in 1:nrow(df)){  
    if(df[i,1] > 50 && df[i,2] < 20){  
      errors[[k]] <- c(row=i, col=2)  
      k <- k + 1  
    }  
  }  
  errors  
}
```

Task 15

Design a recursive calculation system to handle greatest common divisor (GCD) operations for pairs of numbers. Ensure the system includes a foundational exit branch checking for zero match states to avoid infinite loop call-stack crashes.

```
gcd_recursive <- function(a, b){  
  if(b == 0) return(a)  
  gcd_recursive(b, a %% b)  
}
```

Task 16

Create an entry gate authorization workflow function `verify_credentials`. It must process single user attributes using short-circuit logical chains (`&&` and `||`) to avoid running heavy downstream verification functions if early boolean parameters fail.

```
verify_credentials <- function(user_ok, pass_ok, heavy_check){  
  if(user_ok && pass_ok && heavy_check())  
    TRUE  
  else  
    FALSE  
}
```

Task 17

Design an tracking sequence using a while loop that handles text transformations. The loop must concatenate a specific suffix string onto a text variable during each turn, terminating instantly if the character count of the variable exceeds a threshold limit of 150 characters.

```
text <- ""
while(nchar(text) <= 150){
  text <- paste0(text, "_suffix")
}
print(text)
```

Task 18

Build a data frame summary structure that produces a composite analysis object. The processing function must calculate row counts, column types, and unique entry summaries, packing these different data types into a structured summary list that assigns human-readable names to every component slot.

```
summary_df <- function(df){
  list(
    rows = nrow(df),
    column_types = sapply(df, class),
    unique_values = sapply(df, function(x) length(unique(x)))
  )
}
```

Task 19

Design an array processing routine that emulates pointer-free immutability tracking. The function should receive a vector, append a modification tracking indicator flag element onto it, and return the newly extended array vector without altering the state of the original object in the parent workspace.

```
track_vector <- function(v){
  c(v, "Modified")
}
```

Task 20

Create a compound conditional pricing workflow function calculate_tariff. The function must check an input age category and usage bracket using nested if-else blocks, mapping the output directly to a scalar currency charge value based on the combination of brackets.

```

calculate_tariff <- function(age, usage){
  if(age < 18){
    if(usage < 100)
      100
    else
      150
  } else {
    if(usage < 100)
      200
    else
      300
  }
}

```

Task 21

Design a sequence pattern detector function `find_triplets`. The system must loop through a numeric vector to identify any positions where three consecutive numbers follow an strictly increasing progression ($(x_i < x_{i+1} < x_{i+2})$), logging the discovery index positions into a collection vector.

```

find_triplets <- function(x){
  pos <- c()
  for(i in 1:(length(x)-2)){
    if(x[i] < x[i+1] && x[i+1] < x[i+2]){
      pos <- c(pos, i)
    }
  }
  pos
}

```

Task 22

Create a dynamic functional layout wrapper called `execute_with_reporting`. The function must accept an operational function object along with its dataset parameters. It must execute the passed function parameter internally, compute performance durations, and return a list containing both the function's calculated results and temporal analytics.

```

execute_with_reporting <- function(fun, ...){
  start <- Sys.time()
  result <- fun(...)
  end <- Sys.time()
}

```

```
list(
  result = result,
  start_time = start,
  end_time = end,
  duration = end - start
)
```

Task 23

Design a data transformation function `normalize_scores` that takes a matrix. It must calculate column-specific standard deviations. If a column exhibits zero variance, the function must intercept this condition early using a guard clause to prevent division-by-zero errors, returning a pre-assigned baseline vector instead.

```
normalize_scores <- function(mat){
  out <- mat
  for(i in 1:ncol(mat)){
    s <- sd(mat[, i])
    if(s == 0)
      return(rep(0, nrow(mat)))
    out[, i] <- (mat[, i] - mean(mat[, i])) / s
  }
  out
}
```

Task 24

Build a functional configuration manager object. Initialize an engine configuration as a named list where individual entries correspond to operating settings. Write a management function that takes this list structure, validates structural updates against rules, and returns a sanitized revision of the configuration list.

```
config <- list(mode="AUTO", speed=100, enabled=TRUE)
```

```
manage_config <- function(cfg){
  if(cfg$speed < 0) cfg$speed <- 0
  if(is.null(cfg$enabled)) cfg$enabled <- TRUE
  cfg
}
```

Task 25

Design a recursive string processing utility function named `unpack_strings`. The function must accept a nested list structure containing deeply nested character variables and flatten it, extracting individual characters sequentially through recursive calls into a unified flat character array.

```
unpack_strings <- function(x){  
  if(!is.list(x))  
    return(strsplit(as.character(x), "")[[1]])  
  
  res <- character()  
  for(i in x)  
    res <- c(res, unpack_strings(i))  
  
  res  
}
```

Task 26

Create a truth-table checking function `verify_truth_matrix`. The routine must evaluate two logical vectors across all vector positions using an element-wise logical exclusive-OR routine built out of standard primitives (`&`, `|`, `!`), outputting a final logical vector tracking positional matches.

```
verify_truth_matrix <- function(a, b){  
  (a | b) & !(a & b)  
}
```

Task 27

Design an interactive simulation control loop using a repeat block. The system must process rows from an experimental tracking data frame sequentially, updating state vectors until it encounters an anomaly code or reaches the physical row limit of the dataset.

```
i <- 1  
state <- c()  
  
repeat{  
  if(i > nrow(df)) break  
  state <- c(state, df[i, ])  
  if(df$anomaly[i]) break  
  i <- i + 1  
}
```

Task 28

Build a diagnostic parsing module for server message tracking strings. Create a function that reads single lines of log text. It must apply a string-matching switch statement to categorize the message as "INFO", "WARNING", or "ERROR", returning a standardized numerical classification code.

```
parse_log <- function(msg){  
  switch(msg,  
    INFO = 1,  
    WARNING = 2,  
    ERROR = 3,  
    0)  
}
```

Task 29

Design a functional layout for an iterative array expansion pipeline. The function must accept a vector and double its length during every processing cycle by appending localized variance values. Implement an explicit escape constraint checking for size boundaries to avoid system memory exhaustion.

```
expand_vector <- function(v, limit=1000){  
  while(length(v) < limit){  
    v <- c(v, var(v))  
    if(length(v) >= limit) break  
  }  
  v  
}
```

Task 30

Create a mathematical simulation routine `compute_factorial_loop` that avoids recursion entirely. Use a standard bounded iterative loop to multiply falling integer sequences sequentially, ensuring the loop correctly evaluates edge cases like inputs of `\(0\)` or `\(1\)`.

```
compute_factorial_loop <- function(n){  
  if(n == 0 || n == 1) return(1)  
  
  ans <- 1  
  for(i in 2:n){  
    ans <- ans * i  
  }  
}
```

```

    }
    ans
  }

```

Task 31

Design a parameter assignment framework for a custom graph-plotting utility function. The system must support optional default parameters for title strings, color profiles, and scaling boundaries, using internal conditional assessments to determine if the arguments should fall back to defaults or use runtime definitions.

```

plot_graph <- function(data,
                        title = "Graph",
                        color = "blue",
                        xlim = NULL,
                        ylim = NULL){
  if(is.null(xlim)) xlim <- range(data)
  if(is.null(ylim)) ylim <- range(data)
  plot(data, main=title, col=color, xlim=xlim, ylim=ylim)
}

```

Task 32

Build a complex metric extraction module that evaluates financial timeseries data frames. The module must calculate rolling returns, maximum drawdown values, and trailing volatility limits, returning these three distinct datasets packed together within a single organized list structure.

```

financial_metrics <- function(df){
  returns <- diff(df$Price) / head(df$Price, -1)
  drawdown <- cummax(df$Price) - df$Price
  volatility <- sd(returns)

  list(
    returns = returns,
    max_drawdown = max(drawdown),
    volatility = volatility
  )
}

```

Task 33

Design an element isolation module that targets specific rows inside a data frame based on compound criteria. Write a conditional tracking block that filters records where an asset class

matches "Equity" and transaction volumes fall within a specific range, utilizing strict vectorised logic operators.

```
filter_equity <- function(df){  
  df[df$Asset == "Equity" &  
    df$Volume >= 100 &  
    df$Volume <= 1000, ]  
}
```

Task 34

Create a state transition engine using a while loop to simulate queue processing. The loop must remove records one-by-one from a simulated processing queue array list, transforming data structures step-by-step and running continuously until the tracking array becomes entirely empty.

```
process_queue <- function(queue){  
  while(length(queue) > 0){  
    print(queue[[1]])  
    queue <- queue[-1]  
  }  
}
```

Task 35

Design an isolated context processing module that mimics modification tracking without memory side effects. The function must take an input matrix, perform scaling operations on its values, and output the transformed matrix object, leaving the original matrix structure intact in the calling function environment.

```
scale_matrix <- function(mat){  
  result <- mat * 2  
  result  
}
```

Task 36

Build a recursive mathematical sequence calculator that evaluates custom polynomial progressions. The function must calculate sequence updates based on the values of the two preceding step iterations, incorporating a base evaluation condition for early steps to ensure stable progression.

```
poly_sequence <- function(n){  
  if(n <= 2) return(1)  
  poly_sequence(n-1) + poly_sequence(n-2)  
}
```

```
}
```

Task 37

Design a flexible summary routing dispatcher function. The function must accept a dataset alongside a string command parameter (e.g., "summary", "variance", "distribution"), routing the target data to specific analytical subroutines using a character-matching control structure.

```
route_summary <- function(data, command){  
  switch(command,  
    summary = summary(data),  
    variance = var(data),  
    distribution = table(data),  
    "Invalid Command")  
}
```

Task 38

Create an iterative checker routine that steps through a character vector. The loop must evaluate each text string value individually, terminating the entire search sequence immediately using a break keyword if it detects a specific forbidden text string value.

```
check_strings <- function(vec, forbidden){  
  for(i in seq_along(vec)){  
    if(vec[i] == forbidden)  
      break  
    print(vec[i])  
  }  
}
```

Task 39

Design a compound conditional function qualify_loan that takes transaction values. The function must evaluate credit risk parameters using a nested logical structure, ensuring that missing data points (NA) are caught early by explicit validation checks to avoid evaluating incomplete records.

```
qualify_loan <- function(income, score){  
  if(any(is.na(c(income, score))))  
    return("Incomplete Data")  
  
  if(income > 50000){  
    if(score > 700)
```

```

    "Approved"
  else
    "Review"
} else {
  "Rejected"
}
}

```

Task 40

Build an operational pipeline list that aggregates separate data transform stages. The system must group individual data manipulation functions inside a single named tracking list, enabling developers to run specific transform functions sequentially by accessing their list positions.

```

pipeline <- list(
  add = function(x) x + 1,
  square = function(x) x^2,
  double = function(x) x * 2
)

```

```

run_pipeline <- function(x){
  for(f in pipeline){
    x <- f(x)
  }
  x
}

```

Task 41

Design a custom numerical matrix transposition loop structure. The script must iterate through coordinate indices using a nested loop framework, remapping row positions to column slots manually without calling any native matrix transposition commands.

```

transpose_matrix <- function(mat){
  r <- nrow(mat)
  c <- ncol(mat)
  out <- matrix(0, c, r)

  for(i in 1:r){
    for(j in 1:c){
      out[j, i] <- mat[i, j]
    }
  }
}

```

```

    }
    out
}

```

Task 42

Create a dynamic tracking framework for default function parameter evaluation. Write a function where a secondary parameters array dynamically copies the length and structure of the primary data vector if no explicit configuration argument is passed during initialization.

```

track_data <- function(x, params = vector(mode = mode(x), length = length(x))) {
  list(data = x, params = params)
}

```

Task 43

Design a data isolation verification tool to demonstrate R's copy-on-write behavior. Write a function that accepts a heavy data frame, modifies a specific value coordinate internally, and verifies that the memory address reference changes inside the scope while the external parent object remains unmodified.

```

verify_copy <- function(df) {
  temp <- df
  temp[1,1] <- "Modified"
  temp
}

```

Task 44

Build a recursive tree search engine that looks for target values inside a nested list organization. The function must evaluate each node component, continuing down the tree structure recursively if the element is a list, or returning the index coordinate if it finds a match.

```

search_tree <- function(tree, target) {
  if(!is.list(tree)) {
    return(tree == target)
  }

  for(i in seq_along(tree)) {
    if(search_tree(tree[[i]], target))
      return(i)
  }
}

```

```
    return(NULL)
}
```

Task 45

Design a logical status verification system that evaluates multiple diagnostic files. The loop must check vector status metrics across folders, using non-short-circuiting logical operations to build a complete summary tracking log that flags any failing systems.

```
check_systems <- function(status1, status2){
  status1 & status2
}
```

Task 46

Create a robust conditional engine that checks complex multi-level data fields. The function must route data samples using an if-else block that evaluates whether an input value falls within specific numeric ranges, assigning a classification label string to each range.

```
classify_value <- function(x){
  if(x < 0){
    "Negative"
  } else if(x < 50){
    "Low"
  } else if(x < 100){
    "Medium"
  } else{
    "High"
  }
}
```

Task 47

Design a continuous processing data loop using a repeat structure. The loop must read streaming log events from a data repository, modify the event strings, and write them to an archive file, terminating execution only when it detects a specific completion signal.

```
repeat{
  event <- get_next_event()

  if(event == "END")
    break

  event <- paste0(event, "_processed")
}
```

```

write(event, "archive.txt", append = TRUE)
}

```

Task 48

Build a complex mathematical data builder function that groups calculations. The function must compute matrix eigenvalues, row averages, and column range profiles, combining these separate structures into a unified, named list output for downstream analytics.

```

analyze_matrix <- function(mat){
  list(
    eigenvalues = eigen(mat)$values,
    row_average = rowMeans(mat),
    column_range = apply(mat, 2, range)
  )
}

```

Task 49

Design a list-based functional execution router tool. The wrapper function must accept a numerical target array along with a selector string, matching the selector against a named list of operational functions to execute the target transformation.

```

operations <- list(
  square = function(x) x^2,
  cube = function(x) x^3,
  double = function(x) x*2
)

execute_operation <- function(x, op){
  operations[[op]](x)
}

```

Task 50

Create a safe data validation system that screens data vector pipelines. The validation rule must look for missing entries (NA) using a vectorised comparison structure, replacing any missing instances with a baseline value without halting the wider script execution.

```

replace_na <- function(x, value = 0){
  x[is.na(x)] <- value
  x
}

```

```
}
```

Task 51

Design a custom matrix calculation loop that computes localized spatial filters. The script must move across image pixel matrices using nested loops, computing updated cell values from surrounding coordinates while handling border constraints safely through index validation.

```
spatial_filter <- function(mat){  
  out <- mat  
  r <- nrow(mat)  
  c <- ncol(mat)  
  
  for(i in 2:(r-1)){  
    for(j in 2:(c-1)){  
      out[i,j] <- mean(mat[(i-1):(i+1), (j-1):(j+1)])  
    }  
  }  
  out  
}
```

Task 52

Create a dynamic argument calculator function `initialize_bounds`. The function must take a primary vector and an optional threshold parameter that defaults to the mean value of the input vector plus twice its standard deviation, evaluated lazily at runtime.

```
initialize_bounds <- function(x,  
                             threshold = mean(x) + 2 * sd(x)){  
  threshold  
}
```

Task 53

Design an immutable tracking function that manages data state transitions. The utility must take a system state list object, add an operational log record to it, and return the updated state list structure without relying on reference updates or external side effects.

```
update_state <- function(state){  
  state$log <- c(state$log, paste("Updated:", Sys.time()))  
  state  
}
```

Task 54

Build a recursive mathematical function to calculate values along a sequence of numbers. The routine must calculate current step values by combining results from previous structural iterations, defining clear base conditions for early steps to ensure accurate termination.

```
sequence_calc <- function(n){  
  if(n <= 2)  
    return(1)  
  
  sequence_calc(n-1) + sequence_calc(n-2)  
}
```

Task 55

Design a vector evaluation pipeline that screens metrics data frames. The processing logic must check multiple data rows using short-circuiting logical operators (&& and ||) to stop validation checks early if a row fails critical early steps.

```
check_metrics <- function(a, b){  
  result <- logical(nrow(a))  
  
  for(i in 1:nrow(a)){  
    result[i] <- (a[i,1] > 0) && (b[i,1] > 0)  
  }  
  
  result  
}
```

Task 56

Create a robust data reclassification router function using a switch layout. The function must accept character data codes and return corresponding descriptive category strings, falling back to a generic classification catch-all option if no direct match is found.

```
classify_code <- function(code){  
  switch(code,  
    A = "Category A",  
    B = "Category B",  
    C = "Category C",  
    "Unknown Category")  
}
```

Task 57

Design an tracking loop using a while statement to simulate asset depreciation. The loop must reduce an asset's valuation by a fixed percentage during each processing cycle, running continuously until the tracked valuation drops below a target minimum value threshold.

```
depreciate_asset <- function(value, rate = 0.1, minimum = 100){  
  while(value > minimum){  
    value <- value * (1 - rate)  
  }  
  value  
}
```

Task 58

Build a multi-dimensional summary list constructor function. The system must process an environmental tracking data frame, calculate temporal averages, maximum thresholds, and baseline variation counts, wrapping the distinct summaries inside a named data list.

```
environment_summary <- function(df){  
  list(  
    average = sapply(df, mean, na.rm = TRUE),  
    maximum = sapply(df, max, na.rm = TRUE),  
    variation = sapply(df, var, na.rm = TRUE)  
  )  
}
```

Task 59

Design a functional orchestration framework that groups discrete operation steps inside a named list. Write an optimization script that extracts these step functions sequentially, processing an input vector through each step transformation.

```
steps <- list(  
  add = function(x) x + 1,  
  double = function(x) x * 2,  
  square = function(x) x^2  
)
```

```
run_steps <- function(x){  
  for(f in steps){  
    x <- f(x)  
  }  
}
```

```
x  
}
```

Task 60

Create an element-wise logical comparison filter that targets vector data entries. The conditional code block must analyze entry parameters using a single vectorised logical operator, generating a clean logical tracking vector that flags valid records.

```
validate_entries <- function(x){  
  x > 0  
}
```

Task 61

Design a custom looping routine that searches character vectors for target pattern configurations. The loop must examine text elements step-by-step, skipping specific index positions using the next statement if the items match a temporary ignore criteria list.

```
search_pattern <- function(vec, target, ignore){  
  found <- c()  
  
  for(i in seq_along(vec)){  
    if(vec[i] %in% ignore)  
      next  
  
    if(vec[i] == target)  
      found <- c(found, i)  
  }  
  
  found  
}
```

Task 62

Create a dynamic default argument initialization module for an analytical summary function. The function signature must support an optional evaluation matrix that defaults to a identity matrix matched to the dimensions of the primary data input vector.

```
summary_function <- function(x,  
                             eval_matrix = diag(length(x))){  
  list(  

```

```

    data = x,
    matrix = eval_matrix
  )
}

```

Task 63

Design an isolation assessment function that demonstrates pointer-free data flow architectures. The function must accept a named configuration list, update an internal system value, and return the modified list container while leaving the parent variable unchanged.

```

update_config <- function(config){
  config$system_value <- "Updated"
  config
}

```

Task 64

Build a recursive sorting or grouping utility function that processes array segments. The function must partition an input numeric vector into sub-components, calling itself recursively on each sub-component until it reaches a base condition where segments contain only single entries.

```

recursive_sort <- function(x){
  if(length(x) <= 1)
    return(x)

  pivot <- x[1]
  left <- recursive_sort(x[x < pivot])
  right <- recursive_sort(x[x > pivot])

  c(left, pivot, right)
}

```

Task 65

Design a conditional data-checking gate function `verify_sensor_matrix` that takes a multi-column vector array. The function must evaluate rows using compound vectorised operators, generating a clean logical map that isolates coordinates containing anomalies.

```

verify_sensor_matrix <- function(mat){
  (mat < 0) | is.na(mat)
}

```

Task 66

Create an iterative text concatenation loop using a repeat structure. The loop must pull character data rows from an input table, combine them into a single formatted output block, and break out of the loop block when it runs out of input data records.

```
combine_text <- function(tbl){  
  i <- 1  
  out <- ""  
  
  repeat{  
    if(i > nrow(tbl))  
      break  
  
    out <- paste(out, tbl[i,1])  
    i <- i + 1  
  }  
  
  out  
}
```

Task 67

Design a data analysis function that returns a complex nested list structure. The function must calculate frequency distributions, variance boundaries, and outlier tracking vectors, returning them all inside a single list container with clear, human-readable slot names.

```
analyze_data <- function(x){  
  list(  
    frequency = table(x),  
    variance = var(x),  
    outliers = x[x > mean(x) + 2 * sd(x)]  
  )  
}
```

Task 68

Build a routing mechanism using a named list of functions to process varying data columns. The dispatch script must check column header classes and pass the target vectors into corresponding processing functions extracted directly from the list layout.

```
processors <- list(  
  numeric = function(x) mean(x),
```

```

character = function(x) toupper(x),
logical = function(x) any(x)
)

```

```

process_column <- function(col){
  processors[[class(col)]](col)
}

```

Task 69

Design a robust error-checking condition inside a data cleaning script. The validation statement must check input metrics vectors for missing data points (NA) using vectorised logical conditions, ensuring the script runs safely without hitting missing value evaluation errors.

```

safe_check <- function(x){
  !is.na(x) & (x > 0)
}

```

Task 70

Create an iterative matrix row calculation loop. The routine must step through each row of a matrix using a bounded loop layout, calculating a row summary score and updating a tracking array index, while avoiding any explicit pointer modifications.

```

row_summary <- function(mat){
  scores <- numeric(nrow(mat))

  for(i in 1:nrow(mat)){
    scores[i] <- sum(mat[i, ])
  }

  scores
}

```

Task 71

Design a matrix border padding function using a nested loop framework. The system must create an expanded matrix space and copy original cell data into the center coordinates, manually populating the surrounding border zones with zero values based on coordinate boundary checks.

```

border_padding <- function(mat){
  r <- nrow(mat)
  c <- ncol(mat)

```

```

out <- matrix(0, r + 2, c + 2)

for(i in 1:r){
  for(j in 1:c){
    out[i + 1, j + 1] <- mat[i, j]
  }
}

out
}

```

Task 72

Create a functional argument schema for a custom simulation tool `run_simulation`. The configuration signature must accept a parameter matrix that defaults to an element map computed dynamically from a global baseline dataset variable if no runtime matrix is specified.

```

run_simulation <- function(param_matrix = baseline_dataset){
  param_matrix
}

```

Task 73

Design a clear tracking module that showcases copy-on-write functional architecture. The script must track memory usage changes before, during, and after an internal data frame column update operation inside a custom function block.

```

track_memory <- function(df){
  before <- object.size(df)

  temp <- df
  temp[,1] <- temp[,1] + 1

  after <- object.size(temp)

  list(before = before,
        after = after,
        data = temp)
}

```

Task 74

Build a recursive reduction function that simplifies a numeric vector down to a single value. The routine must combine pairs of elements step-by-step, calling itself on the reduced array until it reaches a single scalar base case.

```
reduce_vector <- function(x){
  if(length(x) == 1)
    return(x)

  new <- c()
  for(i in seq(1, length(x)-1, by = 2)){
    new <- c(new, x[i] + x[i+1])
  }

  if(length(x) %% 2 == 1)
    new <- c(new, tail(x,1))

  reduce_vector(new)
}
```

Task 75

Design a logical processing pipeline that filters system log records. The evaluation framework must scan column fields using element-wise logical operations (& and |) to build a clean logical index mask that isolates problematic system records.

```
filter_logs <- function(df){
  (df$status == "FAIL") & (df$error > 0)
}
```

Task 76

Create a nested data validation module using an if-else control structure. The module must inspect incoming data payloads, sorting records into distinct category channels based on whether their values fall inside or outside target engineering specifications.

```
validate_payload <- function(x){
  if(x >= 0 && x <= 50){
    "Normal"
  } else if(x <= 100){
    "Warning"
  } else{
```

```

    "Critical"
  }
}

```

Task 77

Design an tracking tracking loop using a while statement to monitor an optimization routine. The loop must adjust optimization tuning vectors during each cycle, running continuously until successive calculations fall within a target error margin.

```

optimize <- function(error = 100, rate = 0.5){
  while(error > 0.001){
    error <- error * rate
  }
  error
}

```

Task 78

Build a data analysis layout function that outputs a structured list. The function must process a raw telemetry data frame and calculate min, max, and variance profiles, organizing these calculated outputs into a named list container.

```

telemetry_summary <- function(df){
  list(
    minimum = sapply(df, min),
    maximum = sapply(df, max),
    variance = sapply(df, var)
  )
}

```

Task 79

Design a functional strategy pipeline using a named list container. Store three separate data normalization algorithms as functions inside the list, and write a selection wrapper that calls the correct normalization step based on an input string name.

```

normalizers <- list(
  minmax = function(x) (x - min(x)) / (max(x) - min(x)),
  zscore = function(x) (x - mean(x)) / sd(x),
  decimal = function(x) x / 10^nchar(as.character(max(abs(x))))
)

```

```
normalize_data <- function(x, method){
  normalizers[[method]](x)
}
```

Task 80

Create a safe data-cleaning step that screens input vectors for missing values. The validation logic must scan data matrices using vectorised logical operators, substituting any found NA items with zero values without interrupting the wider application pipeline.

```
normalizers <- list(
  minmax = function(x) (x - min(x)) / (max(x) - min(x)),
  zscore = function(x) (x - mean(x)) / sd(x),
  decimal = function(x) x / 10^nchar(as.character(max(abs(x))))
)
```

```
normalize_data <- function(x, method){
  normalizers[[method]](x)
}
```

Task 81

Design a custom numerical matrix sorting routine using nested loops. The script must compare adjacent cell elements across rows and columns step-by-step, rearranging value ordering manually without calling any native sorting functions.

```
sort_matrix <- function(mat){
  r <- nrow(mat)
  c <- ncol(mat)

  for(i in 1:r){
    for(j in 1:(c-1)){
      for(k in 1:(c-j)){
        if(mat[i,k] > mat[i,k+1]){
          temp <- mat[i,k]
          mat[i,k] <- mat[i,k+1]
          mat[i,k+1] <- temp
        }
      }
    }
  }
  mat
}
```

```
}
```

Task 82

Create a flexible configuration argument system for an account matching tool. The function signature must feature an optional character mapping array that defaults to a standardized alphanumeric dictionary list if left unassigned during initialization.

```
match_account <- function(data,
                           mapping = c(letters, LETTERS, 0:9)){
  list(
    data = data,
    mapping = mapping
  )
}
```

Task 83

Design an isolated context data manager function that highlights R's pointer-free architecture. The utility must take a database connection metadata list, insert a session tracking ID string internally, and return the modified list structure without altering the parent instance scope.

```
update_metadata <- function(meta){
  meta$session_id <- paste0("SID_", sample(1000:9999,1))
  meta
}
```

Task 84

Build a recursive tree traversal function that counts matching nodes across an organizational data hierarchy. The function must inspect current node attributes and call itself to look down child branches, returning a cumulative count once it hits leaf boundaries.

```
count_nodes <- function(tree, target){
  if(!is.list(tree)){
    return(as.integer(tree == target))
  }

  total <- 0
  for(node in tree){
    total <- total + count_nodes(node, target)
  }
}
```

```
total
}
```

Task 85

Design a logical gate function that evaluates clinical trial data columns. The check script must evaluate separate diagnostic vectors using short-circuit logical operators (&& and ||) to ensure patient safety thresholds are met before advancing to more resource-heavy analytics.

```
check_trial <- function(temp_ok, bp_ok, heavy_check){
  temp_ok && bp_ok && heavy_check()
}
```

Task 86

Create a robust categorical mapping module using a switch statement. The module must read error codes from an active server log file and map them directly to clear descriptive status strings, handling unmapped codes using a generic fallback description.

```
map_error <- function(code){
  switch(code,
    E100 = "Database Error",
    E200 = "Network Error",
    E300 = "Server Error",
    "Unknown Error")
}
```

Task 87

Design an iterative processing stream loop using a repeat block. The system must fetch data packets from an external source array, modify packet values, and save them to a storage list, breaking out of the loop block only when it encounters an end-of-stream indicator.

```
storage <- list()
i <- 1

repeat{
  packet <- source[i]

  if(packet == "END")
    break

  storage[[length(storage)+1]] <- paste0(packet, "_done")
}
```

```
i <- i + 1
}
```

Task 88

Build an asset valuation summary function that returns a complex named list. The module must calculate current valuation, historical variance boundaries, and projected growth trends from an inventory table, organizing the outputs into a single list container.

```
asset_summary <- function(df){
  list(
    valuation = sum(df$Value),
    variance = var(df$Value),
    growth = mean(df$Growth)
  )
}
```

Task 89

Design a list-based data transform router system. Define an operational list containing data formatting functions, and build a master execution pipeline that takes an input data frame and runs it through the formatting functions sequentially using dynamic list indexing.

```
formatters <- list(
  trim = function(x) trimws(x),
  upper = function(x) toupper(x),
  lower = function(x) tolower(x)
)
```

```
format_pipeline <- function(df){
  for(f in formatters){
    df <- f(df)
  }
  df
}
```

Task 90

Create a safe condition check step within a financial calculator function. The logic must evaluate whether transaction values contain invalid missing points (NA) using a vectorised logical statement, safeguarding downstream calculations against missing value errors.

```
safe_finance <- function(x){
```

```

if(any(is.na(x)))
  x[is.na(x)] <- 0

x
}

```

Task 91

Design a custom spatial matrix pooling routine using nested loops. The script must iterate across large data frames, calculating maximum localized value blocks within designated sub-grid boundaries and writing results to a separate compressed matrix structure.

```

max_pooling <- function(mat, pool = 2){
  r <- nrow(mat)
  c <- ncol(mat)
  out <- matrix(0, floor(r/pool), floor(c/pool))

  rr <- 1
  for(i in seq(1, r-pool+1, by=pool)){
    cc <- 1
    for(j in seq(1, c-pool+1, by=pool)){
      out[rr, cc] <- max(mat[i:(i+pool-1), j:(j+pool-1)])
      cc <- cc + 1
    }
    rr <- rr + 1
  }
  out
}

```

Task 92

Create a dynamic argument framework for a time-series forecasting model function. The signature must support a baseline parameter vector that defaults to a trailing trend sequence computed dynamically from the historical length of the primary input data stream.

```

forecast_model <- function(data,
                           baseline = tail(data, min(5, length(data)))){
  list(
    data = data,
    baseline = baseline
  )
}

```

Task 93

Design a validation model that confirms R's pointer-free copy-on-write memory mechanics. Write an analysis function that receives a list container, updates a specific named element internally, and confirms that the original list outside the function keeps its original data state.

```
update_list <- function(lst){  
  temp <- lst  
  temp$status <- "Updated"  
  temp  
}
```

Task 94

Build a recursive calculation routine that evaluates mathematical combinations or permutations. The function must break the target operation down into smaller factorial sub-components, calling itself recursively until it hits a base termination condition of (1) .

```
factorial_recursive <- function(n){  
  if(n <= 1)  
    return(1)  
  n * factorial_recursive(n - 1)  
}
```

```
combinations <- function(n, r){  
  factorial_recursive(n) /  
    (factorial_recursive(r) *  
      factorial_recursive(n - r))  
}
```

Task 95

Design a logical screening function that targets network traffic logs. The validation engine must check package metrics vectors across rows using element-wise logical operations, outputting a boolean index vector that isolates suspicious traffic patterns.

```
screen_traffic <- function(df){  
  (df$Packets > 1000) & (df$Errors > 0)  
}
```

Task 96

Create a layered data routing engine using an if-else block architecture. The routine must inspect customer profile data structures and group profiles into distinct service tiers based on compound evaluations of asset scale and transaction counts.

```
service_tier <- function(asset, transactions){  
  if(asset >= 100000){  
    if(transactions >= 100)  
      "Premium"  
    else  
      "Gold"  
  } else {  
    if(transactions >= 50)  
      "Silver"  
    else  
      "Basic"  
  }  
}
```

Task 97

Design an analytical optimization tracking loop using a while statement. The loop must evaluate mathematical adjustments on an input vector, terminating execution when the difference between successive iterations drops below an error margin threshold.

```
optimize_vector <- function(x, tol = 1e-6){  
  prev <- x  
  
  repeat{  
    curr <- prev / 2  
    if(max(abs(curr - prev)) < tol)  
      break  
    prev <- curr  
  }  
  
  curr  
}
```

Task 98

Build a complex data summary function that bundles distinct metrics into a list container. The system must read raw manufacturing log data frames, calculate failure distributions, average run durations, and operational yields, outputting these summaries as a named list.

```
manufacturing_summary <- function(df){  
  list(  
    failure_distribution = table(df$Failure),  
    average_run_duration = mean(df$RunDuration),  
    operational_yield = mean(df$Yield)  
  )  
}
```

Task 99

Design a functional pipeline tool that utilizes a list collection of function objects. Write an asset processing wrapper that extracts target math functions from the list structure using character indices, executing the functions dynamically on input datasets.

```
functions <- list(  
  square = function(x) x^2,  
  cube = function(x) x^3,  
  double = function(x) x * 2  
)  
  
asset_process <- function(x, method){  
  functions[[method]](x)  
}
```

Task 100

Create a robust data screening rule that validates matrix profiles before calculation. The evaluation code must scan data vectors for missing value records (NA) using a vectorised comparison operator, replacing any missing instances with row medians to ensure safe execution.

```
validate_matrix <- function(x){  
  for(i in 1:nrow(x)){  
    med <- median(x[i, ], na.rm = TRUE)  
    x[i, is.na(x[i, ])] <- med  
  }  
  x  
}
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Logic & Vectorisation	30%	(28–30 Points) • Zero iterative loops where native vectorised operators (&, , %%, ifelse) apply. • Code handles complete vectors in single computational cycles.	(23–27 Points) • Accurate control setups.	(15–22 Points) • Heavy reliance on nested loops (for, while) for basic transformations. • Calculation code scales poorly and experiences significant execution delay.	(0–14 Points) • Confuses vector and scalar logic tracks. • Triggers frequent length mismatch errors or script termination crashes.
Argument & Flow Integrity	20%	(19–20 Points) • Complete use of dynamic lazy evaluation properties for defaults. • Integrates clean early-exit guard clauses to save memory cycles.	(15–18 Points) • Clear parameter layout.	(10–14 Points) • Missing critical default parameters. • Logic fails if a user leaves out an optional calculation parameter during runtime.	(0–9 Points) • Broken function signatures. • Erroneously uses internal assignment operators (<-) to specify signature defaults.
Object Architecture & Returns	20%	(19–20 Points) • Combines multiple metrics groups into a structured named list. • Completely relies on clean, implicit terminal line returns.	(15–18 Points) • Returns a single clean data structure (matrix/data frame) properly.	(10–14 Points) • Unpredictable return types across different internal calculation branches. • Output formats shift between vectors and matrices.	(0–9 Points) • Breaks isolated scope mechanics. • Updates external values using global modifications (<<-) instead of functional returns.

Edge Case Handling	15%	(14–15 Points) • Explicit validation blocks intercept empty inputs (numeric(0)) or missing data structures (NA, NaN, NULL) immediately.	(11–13 Points) • Code processes standard inputs cleanly. • Filters simple missing elements, but unmapped structures occasionally leak into operations.	(8–10 Points) • System crashes when an unexpected NA value is encountered inside strict logical comparisons.	(0–7 Points) • Complete lack of diagnostic checks. • Empty or missing data structures trigger fatal errors or infinite loops immediately.
Recursion & Memory Strategy	15%	(14–15 Points) • Implements rigid mathematical base cases. • Zero copy-on-write memory overhead achieved via pre-allocation or vectorised setups.	(11–13 Points) • Restricts recursive tasks to safe internal depth limits. • Keeps basic iteration boundaries accurate.	(8–10 Points) • Recursion paths lack solid boundary conditions. • Threatens system stack safety if evaluated beyond small data sets.	(0–7 Points) • Infinite recursion due to a missing base case block. • Triggers immediate stack overflow system crashes.